PROGO AI · Complete Technical Documentation

Comprehensive System Architecture, Algorithms & Design Reference

Version 1.0 | May 2026

Table of Contents

---

1. Project Overview

Progo AI is a production-grade, AI-powered Retrieval-Augmented Generation (RAG) platform that allows users to upload documents (PDF, DOCX, JSON, Excel) and interact with them through an intelligent chat interface. The platform supports five specialized AI modes, each with its own carefully engineered prompt and workflow.

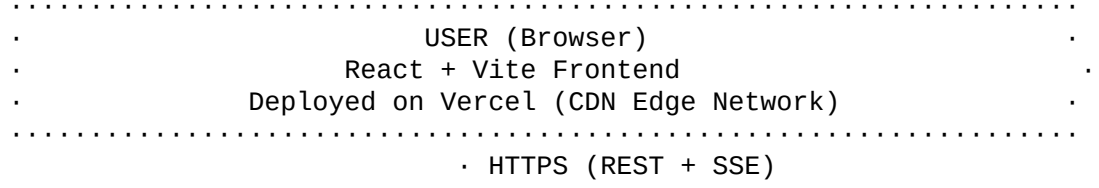
Core Value Proposition

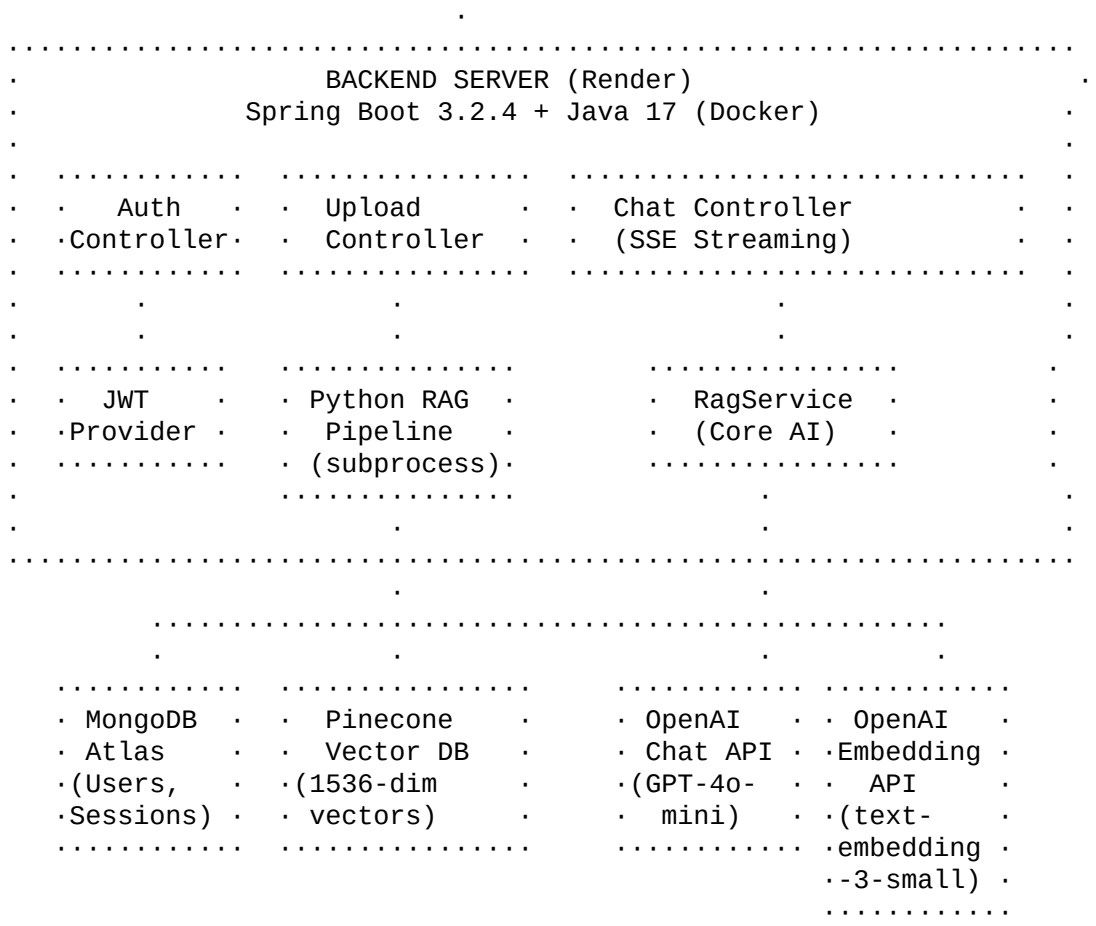
- **Document Q&A:** Upload any document and ask natural language questions; the AI answers strictly from the document content with citations.
- **Mock Interviews:** Paste a Job Description and the AI conducts a structured, scored interview.
- **Interactive Quizzes:** The AI generates MCQ questions on any topic, tracks scores, and adapts difficulty.
- **DSA Code Analysis:** Paste code and get Big-O complexity analysis, bug detection, and optimization suggestions.
- **Simple Chat:** Free-form conversation powered by OpenAI GPT-4o-mini.

---

2. System Architecture

High-Level Architecture Diagram





Data Flow Summary

1. User uploads a document · Backend saves it, triggers Python RAG pipeline
2. Python extracts text · chunks it · generates embeddings · stores in Pinecone
3. User sends a question · Backend embeds the question · queries Pinecone · retrieves relevant chunks
4. Retrieved chunks + question are sent to OpenAI GPT-4o-mini · AI generates answer · streamed to user via SSE

---

3. Technology Stack

Frontend

| Technology | Version | Purpose | |---|---|---| | **React** | 18.x | UI component library | | **Vite** | 5.x | Build tool & dev server (HMR) | | **Tailwind CSS** | 3.x | Utility-first CSS framework | | **Axios** | 1.x | HTTP client with interceptors | | **Lucide React** | Latest | Icon library (tree-shakeable) | | **Vercel** | · | Frontend hosting (CDN + Edge) |

Backend

| Technology | Version | Purpose | |---|---|---| | **Java** | 17 (LTS) | Primary backend language | | **Spring Boot** | 3.2.4 | REST API framework | | **Spring Security** | 6.x | Authentication & authorization | | **Spring Data MongoDB** | 4.x | Database ODM | | **JJWT** | 0.11.5 | JWT token generation & validation | | **Python** | 3.11 | Document ingestion

pipeline | | **Docker** | Multi-stage | Containerization for deployment | | **Render** | · | Backend hosting (Docker container) |

External Services

| Service | Purpose | Model/Tier | |---|---|---| | **OpenAI API** | Chat completions + embeddings | `gpt-4o-mini` + `text-embedding-3-small` | | **Pinecone** | Vector database for similarity search | 1536-dimension index | | **MongoDB Atlas** | User data, chat sessions, file metadata | M0 Free tier |

Python Libraries (for RAG Pipeline)

| Library | Purpose | |---|---| | `openai` | Generating embeddings via OpenAI API | | `tiktoken` | Token counting (for chunking) | | `PyMuPDF (fitz)` | PDF text extraction | | `python-docx` | DOCX/DOC text extraction | | `pandas + openpyxl` | Excel file parsing | | `pinecone` | Vector database upsert operations | | `python-dotenv` | Environment variable management |

---

4. Frontend Architecture

Component Hierarchy

```
App.jsx (Root)
··· ServerWakeupScreen.jsx      (Server cold-start handler)
··· LandingPage.jsx             (Marketing/info page)
··· AuthScreen.jsx              (Login/Register)
··· Main Application Layout
    ··· Sidebar.jsx             (Chat history)
    ·   ··· ChatHistory.jsx
    ·   ··· SessionItem.jsx
    ··· Header (Mode Selector)
    ·   ··· ModeSelector.jsx    (Simple/Q&A/Interview/Quiz/DSA)
    ··· FileUploadPanel.jsx     (Document management)
    ··· ChatArea.jsx            (Message display)
    ·   ··· WelcomeScreen.jsx
    ·   ··· ModeSetupScreen.jsx
    ·   ··· MessageBubble.jsx
    ··· ChatInput.jsx           (User input + file attach)
```

State Management Pattern

The application uses **React Hooks + Context API** (no Redux):

- `AuthContext.jsx` · Global auth state via `createContext` + `useContext`
  - Stores JWT token and userId in `localStorage` for persistence
  - Provides `login()`, `register()`, `logout()` functions
- `useChat.js` (Custom Hook) · Chat session management
  - Manages: `sessions[]`, `currentSessionId`, `messages[]`, `mode`, `isLoading`
  - Uses **Server-Sent Events (SSE)** via `fetch()` + `ReadableStream` for real-time streaming
  - Parses two SSE event types: `metadata` (session info) and `message` (AI content chunks)

- `useFiles.js` (Custom Hook) · File upload management
  - Manages: `uploadedFiles[]`, `activeContext[]`, `isUploading`, `uploadProgress`
  - Uses `FormData` with `multipart/form-data` encoding for file uploads

API Client (`client.js`)

- Built on **Axios** with two interceptors:
  1. **Request Interceptor:** Automatically attaches JWT token from `localStorage` to every request via `Authorization: Bearer <token>` header
  2. **Response Interceptor:** Catches `401/403` errors · clears stored token · forces page reload (auto-logout)

Application Flow State Machine

```
ServerWakeupScreen ··(server responds)··· LandingPage ··(click "Get
Started")··· AuthScreen ··(login)··· Main App
                                                .
                                            .

.................(logout).....................................
```

---

5. Backend Architecture

Controller Layer

`AuthController.java` · `/api/auth`

Endpoint	Method	Purpose	Auth Required
`/api/auth/register`	POST	Create new user account	No
`/api/auth/login`	POST	Authenticate & get JWT	No

Algorithm:

1. Registration: Validate userId (·5 chars) · check uniqueness · hash password with **BCrypt** · save to MongoDB · generate JWT · return token
2. Login: Find user by userId · verify password with BCrypt · generate JWT · return token

`UploadController.java` · `/api/upload`

Endpoint	Method	Purpose	Auth Required
`/api/upload`	POST	Upload files & trigger ingestion	Yes
`/api/upload/files`	GET	List user's uploaded files	Yes

Algorithm:

1. Receive `multipart/form-data` with file(s) and mode
2. For each file: Generate UUID · save to disk · save `FileMetadata` to MongoDB
3. Trigger Python ingestion via `ProcessBuilder` (subprocess):

   ```
   python3 main.py <file_path> <user_id> <file_id>
   ```

4. Create a `ChatSession` with `contextFiles` populated

5 Return `sessionId` to frontend

`ChatController.java` · `/api/chat`

| Endpoint | Method | Purpose | Auth Required | |---|---|---|---| | `/api/chat/{sessionId}/message` | POST (SSE) | Send message to existing session | Yes | | `/api/chat/message` | POST (SSE) | Send message, auto-create session | Yes | | `/api/chat/sessions` | GET | List all user sessions | Yes | | `/api/chat/sessions/{id}` | GET | Get specific session | Yes | | `/api/chat/sessions/{id}` | DELETE | Delete session | Yes | | `/api/chat/sessions/{id}/title` | PUT | Rename session | Yes |

`HealthController.java` · `/api/health`

| Endpoint | Method | Purpose | Auth Required | |---|---|---|---| | `/api/health` | GET | Health check for cold-start detection | No |

Service Layer · `RagService.java`

This is the **brain** of the entire application. It orchestrates:

1 Session management
2 RAG search pipeline
3 Mode-specific prompt engineering
4 OpenAI API calls (both streaming and non-streaming)
5 Pinecone vector search

---

6. RAG Pipeline · The Core Algorithm

Phase 1: Document Ingestion (Python)

Step 1 · Document Loading (`document_loader.py`)

```
Input: Raw file (PDF/DOCX/JSON/Excel)
Output: Structured document with pages[] array
```

| Format | Library | Method | |---|---|---| | PDF | PyMuPDF (`fitz`) | `fitz.open()` · iterate pages · `page.get_text()` | | DOCX/DOC | `python-docx` | `Document()` · join all paragraph texts | Excel | `pandas` + `openpyxl` | `pd.read_excel()` · `df.to_string()` per sheet | | JSON | Built-in `json` | `json.load()` · `json.dumps(indent=2)` |

Document ID Generation: SHA-256 hash of the raw file bytes · deterministic, collision-resistant identifier.

Step 2 · Chunking (`chunk_generator.py`)

Algorithm: Sliding Window Tokenizer-Aware Chunking

```
Parameters:
  - max_tokens = 500 (maximum tokens per chunk)
  - overlap = 50 (tokens of overlap between consecutive chunks)
  - tokenizer = tiktoken.encoding_for_model("gpt-4o-mini")
Algorithm:
  1. Concatenate all page texts into a single token stream
  2. Track page_number for each token position
  3. Slide a window of max_tokens across the stream
  4. Step forward by (max_tokens - overlap) = 450 tokens each
```

```
iteration
  5. Decode tokens back to text for each chunk
  6. Assign chunk_id (UUID v4), document_id, page_start, page_end
```

Why this approach?

- **Token-aware** (not character-based) ensures each chunk fits within the embedding model's context window
- **Overlap of 50 tokens** prevents information loss at chunk boundaries · sentences that span two chunks are captured in both
- **Page tracking** enables source citations ("See page 3")

Step 3 · Embedding Generation (`embedding_generator.py`)

```
Model: OpenAI text-embedding-3-small
Dimension: 1536
Batch Size: 64 chunks per API call (for efficiency)
```

Process:

1. Collect chunk texts in batches of 64
2. Send batch to OpenAI Embeddings API · receive 1536-dimensional vectors
3. Map each vector back to its chunk_id

Step 4 · Vector Storage (Pinecone)

```
Index: "ragtest"
Dimension: 1536
Metric: Cosine Similarity (default)
Batch upsert: 100 vectors per batch
```

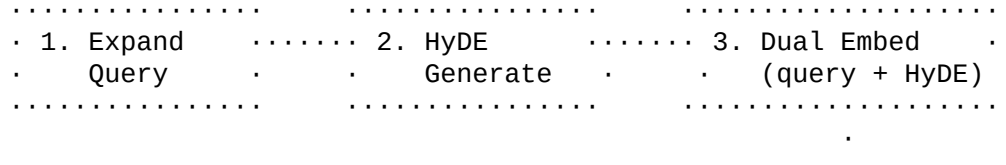
Metadata stored per vector:

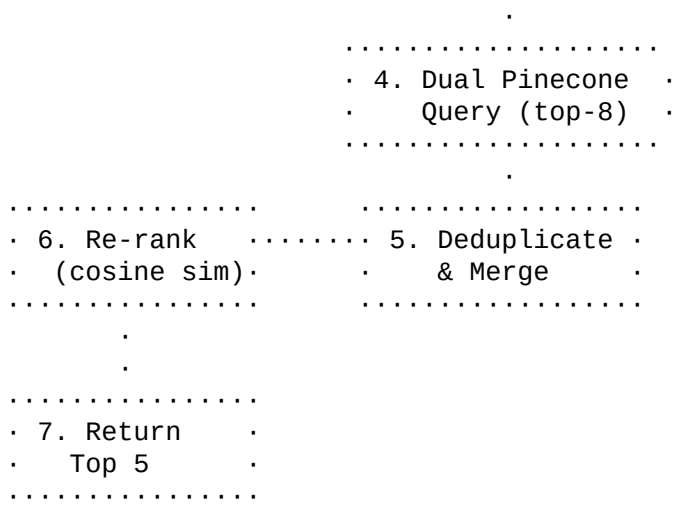
```
{
  "user_id": "vipulagarwal",
  "file_id": "uuid-of-file",
  "document_id": "sha256-hash",
  "text": "actual chunk text content",
  "page_start": 1,
  "page_end": 2,
  "source_type": "pdf",
  "source_name": "document.pdf"
}
```

Phase 2: Query-Time Retrieval (Java · `RagService.java`)

Enhanced Search Pipeline (HyDE + Re-ranking)

This is a **production-grade 7-step retrieval algorithm**:





Step 1 · Query Expansion (LLM-powered)

- Send user's question to GPT-4o-mini with instruction: *"Generate a richer version with synonyms and related terms"*
- Result: broader search coverage (e.g., "What is ML?" ·"What is machine learning, artificial intelligence, neural networks, deep learning?")

Step 2 · HyDE (Hypothetical Document Embedding)

- Send user's question to GPT-4o-mini with instruction: *"Write a short paragraph that would be the ideal answer found in documentation"*
- **Key Insight:** Instead of searching for the question, we search for what the answer looks like. This finds semantically closer matches in the vector space.

Step 3 · Dual Embedding

- Embed the original query + expanded query together ·`queryEmbedding`
- Embed the HyDE hypothetical answer ·`hydeEmbedding`

Step 4 · Dual Pinecone Query

- Query Pinecone with `queryEmbedding` · get top-8 results
- Query Pinecone with `hydeEmbedding` · get top-8 results
- Filter by: `user_id` (data isolation) + `source_name` (active context files)

Step 5 · Deduplication

- Merge both result sets using a `LinkedHashMap<chunkId, ScoredChunk>`
- If a chunk appears in both results, keep the one with the higher score

Step 6 · Re-ranking (Custom Cosine Similarity)

```
rerankScore = 0.4 × originalPineconeScore + 0.6 ×
cosineSimilarity(chunkEmbedding, queryEmbedding)
```

- Embed each chunk's text (truncated to 500 chars)
- Compute cosine similarity against the original query embedding
- Weighted combination: 40% Pinecone score + 60% re-rank similarity
- Sort by `rerankScore` descending

Step 7 · Return Top 5

- Take the top 5 re-ranked chunks
- Join with `\n\n---\n\n` separator
- Inject into the system prompt as `RELEVANT DOCUMENTATION CONTEXT`

Cosine Similarity Formula:

```
cos(A, B) = (A · B) / (||A|| × ||B||)
where:
  A · B = ·(a· × b·)   (dot product)
  ||A|| = ·(·(a·²))     (L2 norm)
```

---

7. Authentication & Security

JWT (JSON Web Token) Flow

```
Client                                  Server
  ·                                       ·
  ··· POST /api/auth/login ·············
  ·   {userId, password}                  ·
  ·                                       ··· Validate credentials
  ·                                       ··· BCrypt.matches(plain, hash)
  ·                                       ··· Generate JWT (HS512)
  ···· {token: "eyJ...", userId} ·······
  ·                                       ·
  ··· GET /api/chat/sessions ···········
  ·   Authorization: Bearer eyJ...        ·
  ·                                       ··· JwtAuthFilter extracts token
  ·                                       ··· Validate signature (HS512)
  ·                                       ··· Extract userId from claims
  ·                                       ··· Set SecurityContext
  ···· [sessions array] ···············
```

Security Components

`JwtTokenProvider.java`

- **Algorithm:** HMAC-SHA512 (`HS512`)
- **Key:** Auto-generated 512-bit key via `Keys.secretKeyFor(SignatureAlgorithm.HS512)`
- **Expiry:** 24 hours (86,400,000 ms)
- **Claims:** `sub` (userId), `iat` (issued at), `exp` (expiration)

`JwtAuthenticationFilter.java`

- Extends `OncePerRequestFilter` (Spring Security)
- Extracts JWT from `Authorization: Bearer <token>` header
- Validates token · extracts userId · sets `SecurityContextHolder`

`SecurityConfig.java`

- **Permit all:** `/api/auth/**`, `/api/health/**`

- **Require auth:** Everything else
- **CORS:** `allowedOriginPatterns("*")` to support Vercel deployments
- **CSRF:** Disabled (stateless JWT architecture)
- **Session:** `STATELESS` (no server-side sessions)

Password Hashing:

- **Algorithm:** BCrypt (via Spring Security's `PasswordEncoder`)
- **Strength:** Default work factor of 10 (2^10 = 1024 rounds)
- **Salt:** Automatically generated per password (stored within the hash)

---

8. AI Modes & Prompt Engineering

Mode 1: Simple Chat

```
System Prompt: "You are Progo AI, a helpful, knowledgeable, and
friendly assistant..."
RAG: Disabled
Temperature: 0.7
```

Mode 2: Document Q&A

```
System Prompt: "You are Progo AI operating in strict Document Q&A
mode.
  You MUST answer ONLY based on the provided documentation context.
  Rules:
  - If user is greeting, acknowledge warmly and say you are ready
  - If answer exists in context, provide it with specific
references
  - If answer NOT in context, say 'This information is not found'
  - NEVER make up information from general knowledge
  - Quote relevant passages when possible"
RAG: ENABLED (Enhanced HyDE + Re-ranking pipeline)
Context Injection: "RELEVANT DOCUMENTATION
CONTEXT:\n{retrieved_chunks}"
```

Mode 3: Mock Interview

```
System Prompt: "You are an expert AI Mock Interviewer.
  JOB DESCRIPTION: {pasted_jd}

  PROTOCOL:
  1. Generate ONE question at a time
  2. After user answers: Evaluate (Good/Average/Needs Improvement)
  3. Explain ideal answer (2-3 sentences)
  4. Immediately ask NEXT single question
  5. After 8-10 questions: Show final scorecard (X/10)

  CRITICAL RULES:
  - NEVER ask more than ONE question per message
  - NEVER give a list of questions"
```

Mode 4: Quiz

```
System Prompt: "You are Progo AI Quiz Master.
  TOPIC: {user_input}

  PROTOCOL:
  1. Generate ONE MCQ with 4 options (A/B/C/D)
  2. After user answers: · Correct! or · Incorrect
  3. Brief explanation + running score: 'Score: X/Y (Z%)'
  4. After 10 questions or 'stop': Final scorecard"
```

Mode 5: DSA Code Analysis

```
System Prompt: "You are Progo AI DSA Expert.
  CODE: {pasted_code}

  ANALYSIS:
  1. Identify algorithm and language
  2. Time Complexity: Big-O with step-by-step reasoning
  3. Space Complexity: Big-O with reasoning
  4. Bug detection and edge cases
  5. Optimized approach with code"
```

Auto Title Generation

When the first message is sent, the backend automatically generates a 6-word title using GPT-4o-mini:

```
"Generate a very short title (max 6 words) for a conversation.
Return ONLY the title."
```

---

9. Real-Time Streaming (SSE)

Server-Sent Events Architecture

Why SSE over WebSockets?

- Unidirectional (server · client) · perfect for AI streaming
- Works over standard HTTP/HTTPS · no upgrade handshake needed
- Simpler implementation, auto-reconnect built into browser API
- Compatible with all CDNs and reverse proxies

Implementation

Backend (`RagService.java`):

```
SseEmitter emitter = new SseEmitter(600000L); // 10-minute timeout
// Step 1: Send metadata event (session info)
emitter.send(SseEmitter.event().name("metadata").data(meta));
// Step 2: Stream OpenAI response token-by-token
// Uses java.net.http.HttpClient with streaming body handler
```

```
HttpResponse<Stream<String>> response = client.send(request,
    HttpResponse.BodyHandlers.ofLines());
// Step 3: Parse each SSE line from OpenAI
// Forward content delta to client
emitter.send(SseEmitter.event().name("message").data(chunk));
```

Frontend (`useChat.js`):

```
// Uses fetch() + ReadableStream (not EventSource) for POST support
const reader = response.body.getReader();
const decoder = new TextDecoder();
// Parse SSE format: "event: message\ndata: {...}\n\n"
// Append each content chunk to the last assistant message
```

Data Flow:

```
OpenAI API ··(token)··· Spring Boot ··(SSE event)··· Browser ··(setState)···
React UI
   "Hello"                  event: message              append
"Hello"         re-render
   " world"                 data: {"content":" world"}  append "
world"        re-render
   [DONE]                   emitter.complete()          stream ends
           final state
```

---

10. Deployment Architecture

Docker Multi-Stage Build (`Dockerfile`)

```
# Stage 1: Build Java (Maven)
FROM maven:3.9.6-eclipse-temurin-17 AS build
  · mvn clean package -DskipTests
# Stage 2: Runtime (Java + Python)
FROM eclipse-temurin:17-jre-jammy
  · Install Python 3 + pip + venv
  · Create virtual environment
  · pip install requirements.txt
  · Copy Python scripts
  · Copy built JAR from Stage 1
  · Run: java -jar backend.jar
```

Render (Backend)

- **Service Type:** Docker Web Service
- **Build:** Automatically builds from Dockerfile
- **Environment Variables:** `MONGODB_URI`, `OPENAI_API_KEY`, `PINECONE_API_KEY`, etc.
- **Cold Start:** Free tier instances sleep after 15 min of inactivity · handled by `ServerWakeupScreen`

Vercel (Frontend)

- **Framework:** Vite (auto-detected)
- **Build Command:** `npm run build`

- **Output Directory:** `dist/`
- **Environment Variable:** `VITE_API_URL` · Render backend URL
- **SPA Routing:** `vercel.json` rewrites all routes to `index.html`

Cold-Start Handling

```
User opens site · Frontend polls GET /api/health every 3 seconds
  · If server sleeping: Show ServerWakeupScreen (orbital animation)
  · If server responds: Transition to LandingPage
  · Render wakeup takes ~30-60 seconds on free tier
```

---

11. API Reference

Authentication

```
POST /api/auth/register
Body: { "userId": "string", "password": "string" }
Response: { "token": "jwt_string", "userId": "string" }
POST /api/auth/login
Body: { "userId": "string", "password": "string" }
Response: { "token": "jwt_string", "userId": "string" }
```

File Upload

```
POST /api/upload
Content-Type: multipart/form-data
Body: files[] (binary), mode (string: "qna")
Response: { "message": "...", "sessionId": "..." }
GET /api/upload/files
Response: ["filename1.pdf", "filename2.docx"]
```

Chat

```
POST /api/chat/{sessionId}/message
POST /api/chat/message
Content-Type: application/json
Body: { "message": "string", "mode":
"simple|qna|interview|quiz|dsa" }
Response: text/event-stream (SSE)
GET /api/chat/sessions
GET /api/chat/sessions/{id}
DELETE /api/chat/sessions/{id}
PUT /api/chat/sessions/{id}/title
```

Health

```
GET /api/health
Response: { "status": "UP" }
```

---

12. Database Schema

MongoDB Collections

`users`

```
{
  "_id": "ObjectId",
  "userId": "string (unique)",
  "password": "string (BCrypt hash)"
}
```

`chat_sessions`

```
{
  "_id": "string (auto-generated)",
  "title": "string",
  "mode": "simple | qna | interview | quiz | dsa",
  "userId": "string",
  "createdAt": "ISO 8601 timestamp",
  "messages": [
    { "role": "user | assistant", "content": "string" }
  ],
  "contextFiles": ["filename1.pdf", "filename2.docx"],
  "setupContext": "string (JD, quiz topic, or code)",
  "state": { }
}
```

`files`

```
{
  "_id": "ObjectId",
  "fileId": "UUID string",
  "userId": "string",
  "filename": "string",
  "selectedMode": "string",
  "uploadTime": "ISO 8601 timestamp"
}
```

Pinecone Index

```
Index Name: "ragtest"
Dimension: 1536
Metric: Cosine Similarity
Namespace: default ("")
Vector Metadata Schema:
{
  "user_id": "string",
  "file_id": "string",
  "document_id": "string (SHA-256)",
  "text": "string (chunk content)",
  "page_start": integer,
  "page_end": integer,
  "source_type": "pdf | docx | json | excel",
```

```
  "source_name": "string (filename)"
}
```
